TRIBHUVAN UNIVERSITY

INSTITUTE OF ENGINEERING

THAPATHALI CAMPUS

FIRST PERSON MAZE GAME

SUBMITTED BY:

AMRITA REGMI THA081BEI005

PRAJWAL KHANAL THA081BEI028

SHAKSHI GYAWALI THA081BEI037

PROGRAM: BEI **BATCH: 2081**

YEAR/PART: II/I

SUBJECT: COMPUTER GRAPHICS & VISUALIZATION

SUBMITTED TO:

INSTRUCTOR NAME: ASST. PROF. KOBID KARKEE

DEPARTMENT: ELECTRONICS & COMPUTER ENGINEERING

CHAITRA, 2082

Acknowledgement

I would like to express my sincere gratitude to everyone who supported and guided me throughout the development of the **"First person maze game"** project.

First and foremost, I would like to thank my project supervisor sir **Kobid Karkee** for their constant guidance, insightful feedback, and encouragement throughout the course of this project. Their expertise and support were invaluable in shaping the direction of this work.

I would also like to acknowledge the contributions of my colleagues and friends, who provided assistance and offered constructive suggestions during the development process. Their collaborative spirit and technical support were crucial in overcoming challenges faced during implementation.

Special thanks to the **Department of Electronics and Computer Engineering**, IOE Campus, Thapathali for providing the necessary resources and an excellent learning environment, which enabled me to complete this project successfully.

This project would not have been possible without all of your contributions. Thank you..

Amrita Regmi (Class Roll No: 081BEI005)

Prajwal Khanal (Class Roll No: 081BEI028)

Shakshi Gyawali (Class Roll No: 081BEI037)

Abstract

This project focuses on the design and implementation of a First Person Maze Game using the Godot game engine. The primary objective is to demonstrate fundamental concepts of computer graphics, including 3D environment creation, camera control, collision detection, and basic maze structuring techniques. In this system, the player navigates through a maze from a first-person perspective, aiming to reach the endpoint while a timer records the total completion time. The game integrates interactive controls, real-time rendering, and sound effects to enhance user engagement and create an immersive experience. The project highlights the practical application of theoretical graphics concepts by combining scripting and game engine tools to develop a functional 3D application. Overall, it serves as a learning platform for understanding how computer graphics principles can be effectively used in interactive system design and development.

Keywords: *Computer Graphics, First-Person Game, 3D Visualization*

Table of Contents

Introduction……..……...…1

Objectives……...…….……4

Tools and Technologies Used……...……...…….……5

Methodology…….……...…….……...…6

Implementation……...…….....……..…15

Result……..……..…….20

Discussion…….……..…23

Conclusion…..……....……..…….25

Future Work……..……26

References…….……..…….27

INTRODUCTION

1.1 Background Introduction

Computer Graphics is a specialized field of computer science that deals with the generation, representation, manipulation, and display of visual information using computers. Over the past few decades, computer graphics has evolved from simple two-dimensional drawings to highly realistic three-dimensional interactive environments. Today, computer graphics is widely used in areas such as gaming, virtual reality, medical visualization, engineering simulations, education, entertainment, and scientific research.

At the core of computer graphics lies the concept of representing objects using mathematical models and displaying them on digital screens through rendering techniques. Concepts such as rasterization, transformations, projections, illumination, shading, and animation form the foundation of this field. Modern graphics systems use powerful hardware and software pipelines to process geometric data and convert it into visually meaningful images in real time.

One of the most impactful applications of computer graphics is in interactive 3D environments, particularly video games. Games use computer graphics techniques to create immersive virtual worlds where users can interact with objects in real time. A first-person maze game is a classic example that combines 3D modeling, viewing transformations, collision detection, and real-time rendering to create an engaging experience for the user.

Game engines such as Godot, Unity, and Unreal Engine provide comprehensive frameworks that integrate graphics rendering, physics simulation, input handling, and scripting. These engines allow developers to focus on implementing logic and design while relying on built-in systems for rendering and performance optimization. Godot, being open-source and lightweight, is particularly suitable for educational projects and rapid prototyping.

This project focuses on designing and developing a First-Person Maze Game using the Godot Engine to practically demonstrate fundamental computer graphics concepts taught in the course. The maze environment is constructed using grid-based modeling, and the player navigates through the maze using a first-person camera, experiencing the environment from a realistic viewpoint.

1.2 Motivation

The motivation for this project arises from the need to connect theoretical computer graphics concepts with practical implementation. In classroom learning, students are introduced to algorithms and mathematical foundations such as line drawing algorithms, coordinate transformations, clipping techniques, and projection methods. However, understanding how these concepts work together in real-time interactive systems often remains challenging without hands-on experience.

A first-person maze game provides an ideal platform to apply these concepts in a meaningful way. The development of such a game involves designing 3D objects, positioning them correctly in a coordinate system, applying transformations, setting up a camera for first-person viewing, and ensuring smooth interaction through collision detection and physics handling. This practical exposure enhances conceptual clarity and builds confidence in applying graphics principles.

Additionally, game development encourages creativity, logical thinking, and teamwork. Working as a group allows students to divide tasks such as environment design, scripting, testing, documentation, and presentation. The project also introduces students to modern development tools and workflows, which are highly relevant in the software and gaming industries.

Another motivating factor is the increasing importance of interactive visualization and real-time graphics in emerging technologies such as virtual reality (VR), augmented reality (AR), and simulation-based training systems. By developing a first-person maze game, students gain foundational skills that can be extended to more advanced applications in the future.

1.3 Problem Definition

Despite having a strong theoretical foundation in computer graphics, students often face difficulties in visualizing and implementing concepts in real-world applications. Understanding abstract ideas such as coordinate transformations, camera viewing pipelines, and object-space versus image-space techniques can be challenging without practical examples.

The problem addressed in this project is the lack of practical exposure to implementing a complete interactive 3D system that integrates multiple computer graphics concepts. Specifically, the challenge is to design and implement a 3D maze environment where the player can navigate freely using a first-person perspective while ensuring correct object placement, collision handling, and real-time rendering performance.

The project must ensure that the maze is logically structured, visually consistent, and interactive. Walls and floors must be placed using accurate coordinate mapping, player movement must respect collision boundaries, and the camera must simulate realistic first-person motion. Achieving this requires a clear understanding of 3D coordinate systems, transformations, and rendering pipelines.

Thus, the problem can be defined as developing a functional and interactive first-person maze game that demonstrates the correct application of fundamental computer graphics principles using a modern game engine.

Objectives

The primary objective of this project is to design and develop a First-Person Maze Game that demonstrates the practical application of computer graphics concepts learned during the course. The specific objectives of the project are as follows:

- To apply fundamental computer graphics principles such as 3D coordinate systems, transformations, and projections in a practical environment.
- To implement a first-person camera system that allows realistic navigation within a 3D environment.
- To incorporate collision detection and physics handling to restrict player movement within the maze boundaries.
- To understand and utilize the graphics pipeline provided by a game engine.
- To develop teamwork and collaboration skills through task distribution among group members.
- To document the design, implementation, and outcomes of the project in a structured technical report.
- To enhance presentation and communication skills by explaining the project concepts and results effectively.

Tools and Technologies Used

- Programming Language: GDScript
- Game Engine: Godot Engine
- Development Environment: Godot Editor / VS Code
- Components: 3D Rendering, Physics Engine, Audio System

Methodology

4.1 Introduction

This project focuses on the design and development of a 3D maze exploration game using the Godot game engine. The system integrates multiple components including procedural maze representation, player physics, environment modeling, and interactive gameplay elements.

The methodology follows a modular and collaborative development approach, where each subsystem is independently designed and later integrated into a unified system. This ensures scalability, maintainability, and minimal merge conflicts during team development.

4.2 System Architecture Overview

The system is designed using a **scene-based architecture**, which is a core principle of the Godot engine. Each major component is encapsulated in a separate scene and later composed into a central scene (Main.tscn).

4.2.1 Core Modules

- Maze Generation System
- Player Controller System
- Environment System
- UI & Feedback System
- Audio-Visual Enhancement System

Each module operates independently but communicates through the scene graph.

4.3 Discrete to Continuous Mapping

4.3.1 Concept

The maze is logically represented as a **discrete grid**, but visually rendered in a **continuous 3D space**.

- Discrete Representation:
  A 2D array → maze[row][col]
- Continuous Representation:
  3D coordinates → Vector3(x, y, z)

4.3.2 Mapping Strategy

- Row → Z-axis
- Column → X-axis
- Y-axis → Height (vertical placement)

4.4 Transformation Formula

Position=(col×scale, height, row×scale)\text{Position} = (col \times scale,\ height,\ row \times scale)Position=(col×scale, height, row×scale)

Where:

- scale = size of each grid cell (e.g., 3 units)

4.5 Significance

This mapping ensures:

- Accurate spatial placement
- Easy scalability of maze size
- Logical-to-visual consistency.
- Store maze structure
- Interpret grid values
- Instantiate walls dynamically

4.6 Maze Generation System

4.6.1 Data Structure

e.g. Var maze_grid = [

```
    [1,1,1,1,1],
    [1,0,0,0,1],
    [1,0,1,0,1],
    [1,0,0,0,1],
    [1,1,1,1,1]
]
```

- 1 → Wall
- 0 → Path

4.6.2 Algorithm

1. Iterate through each row and column
2. Check cell value
3. If value == 1:
    - Instantiate wall scene
    - Place in 3D space using mapping formula

4.6.3 Design Justification

- Simple and efficient
- Easy to debug
- Extendable for procedural generation in future

4.7 Player Controller System

4.7.1 Physics-Based Movement

The player is implemented using CharacterBody3D, ensuring:

- Gravity simulation
- Collision handling

- Smooth motion

4.7.2 Input Handling

Movement is controlled using:

Input.get_vector("left", "right", "up", "down")

4.7.3 Directional Transformation

Local input is converted to world direction using:

Direction=Basis×(x,0,y)\text{Direction} = \text{Basis} \times (x, 0, y)Direction=Basis×(x,0,y)

4.7.4 Features

- WASD movement
- Acceleration and velocity smoothing
- Collision-aware navigation

4.8 Mouse Look System

The system separates rotation into:

- **Yaw (Horizontal rotation)** → Player body
- **Pitch (Vertical rotation)** → Camera

4.9 Environment System

4.9.1 Wall Design

Walls are implemented as reusable blueprints:

- StaticBody3D
- MeshInstance3D
- CollisionShape3D

4.9.2 Design Principle: Local vs World Space

Keeping the wall centered at origin:

- Simplifies placement
- Avoids transformation errors

4.10 Floor System

A floor is essential to:

- Prevent infinite falling
- Support physics interactions

4.11 Lighting System

- **DirectionalLight3D** simulates sunlight
- **WorldEnvironment** enhances realism

4.12 Rendering Choice

- Forward+ Renderer selected for high-quality lighting and shadows

4.13 Scene Composition (Main System)

The Main.tscn acts as the **integration layer**.

4.13.1 Design Philosophy

- No logic inside Main
- Only composition of modules

4.13.2 Scene Structure

```
Main
├── Maze
├── Player
├── Floor
├── Light
└── Environment
```

4.13.3 Advantages

- Modular development
- No dependency conflicts
- Easy debugging

4.14 Interaction System

4.14.1 Goal Detection

Implemented using Area3D:

- Detects overlap
- Does not block movement

4.14.2 Win Condition

Triggered when player enters goal area:

- Display win message
- Stop player movement

4.14.3 System Flow

1. Player reaches goal
2. Signal emitted
3. UI updated
4. Physics disabled

4.15 UI System

4.15.1 Timer System

Tracks gameplay duration:

Time=∑Δt\text{Time} = \sum \Delta tTime=∑Δt

Updated every frame using _process(delta).

4.15.2 Minimap System

- Displays top-down maze view
- Activated on key press ("H")
- Shows player position dynamically

4.15.3 Design Benefits

- Enhances navigation
- Provides player assistance
- Improves user experience

4.16 Audio System

4.16.1 Background Music

- Continuous playback
- Enhances immersion

4.16.2 Footstep Audio

- Triggered based on movement
- Stops when player is idle

4.16.3 Design Consideration

- Event-driven sound system
- Optimized for performance

4.17 Visual Enhancement System

4.17.1 PBR Materials

Physically Based Rendering simulates real-world light interaction:

Property	Purpose
Albedo	Base color
Roughness	Surface reflection
Metallic	Material type

4.17.2 Flashlight System

Attached to camera:

- Moves with player view
- Improves navigation in dark areas

4.17.3 Fog & Sky

- Fog adds depth and mystery
- Sky improves environmental realism

4.18 System Integration Strategy

The project follows a **parallel development workflow**:

Member	Responsibility
Shakshi	Maze logic
Prajwal	Player system
Amrita	Environment & visuals

4.19 Integration Approach

- Each module developed independently
- Integrated in Main scene
- Zero overlap in responsibilities

4.20 Advantages of the System Design

- Modular and scalable
- Easy debugging and testing
- Real-time interaction
- High visual realism
- Clear separation of concerns

Implementation

The implementation of the First-Person Maze Game was carried out using a modular approach in the **Godot Engine**, where each subsystem was developed independently and later integrated into the main scene. The project primarily uses **GDScript** for scripting and leverages built-in physics and rendering systems.

5.1 Project Structure

The project is organized into multiple scenes and scripts to ensure modularity and maintainability. The main components include:

- Main Scene (Main.tscn) – Integrates all subsystems
- Maze Scene – Responsible for maze generation and structure
- Player Scene – Handles player movement and camera
- Goal Scene – Detects game completion
- UI Scene – Displays timer and messages

Each scene is associated with scripts that define its behavior.

5.2 Maze Generation Implementation

The maze is implemented using a 2D grid array, where:

- 1 represents walls
- 0 represents walkable paths

The script MazeGenerator.gd iterates through the grid and dynamically instantiates wall objects.

5.2.1 Key Steps:

1. Define the grid structure

2. Loop through rows and columns
3. Check cell value
4. Instantiate wall if value = 1
5. Position wall using coordinate transformation

5.2.2 Position Mapping:

The discrete grid is converted into 3D coordinates using:

- X-axis → Column
- Z-axis → Row

This ensures proper alignment of the maze in 3D space.

5.3 Player Controller Implementation

The player is implemented using CharacterBody3D, which provides built-in physics and collision handling.

5.3.2 Movement System

- Input is taken using keyboard controls (W, A, S, D)
- Movement direction is calculated relative to the player's orientation
- Velocity is updated every frame

5.3.3 Features:

- Smooth movement using velocity interpolation
- Gravity simulation
- Collision-aware navigation

5.4 Camera and Mouse Look System

A first-person camera is attached to the player.

5.4.1 Implementation Details:

- Horizontal rotation (Yaw) → Player body
- Vertical rotation (Pitch) → Camera

5.4.2 Constraints:

- Pitch is clamped between -80° and +80° to prevent unnatural rotation

This creates a realistic first-person viewing experience similar to FPS games.

5.5 Collision and Physics Handling

Collision detection is handled using:

- CollisionShape3D for walls
- Physics engine of Godot

5.5.1 Functionality

- Prevents player from passing through walls
- Ensures realistic movement within the maze

5.6 Goal Detection System

The goal is implemented using Area3D, which detects when the player enters a specific region.

Working Mechanism:

1. Player enters goal area
2. Signal is triggered
3. Game completion event occurs
4. Timer stops and result is displayed

5.7 Timer System

The timer is implemented using a script that updates every frame.

Logic:

- Time is incremented using delta time
- Displayed on screen using UI elements

This allows tracking of player performance.

5.8 Audio Implementation

Audio enhances user experience through:

- Background music (looped)
- Footstep sounds (triggered during movement)

An event-driven approach is used to optimize performance.

5.9 Lighting and Rendering

The scene uses:

- DirectionalLight3D for global lighting

- WorldEnvironment for visual effects

Rendering:

- Forward+ rendering is used for better lighting quality

5.10 System Integration

All modules are integrated into the main scene.

Integration Strategy:

- Each member developed modules independently
- Final integration done in Main scene
- Ensured no conflicts between components

5.11 Limitations

- Static maze (not procedurally generated yet)
- Basic AI (no enemies or dynamic elements)
- Limited optimization for large-scale mazes

Result

The final output of the project is a fully functional interactive 3D first-person maze game developed using the Godot engine.

6.1 Game Functionality

The system successfully demonstrates the following features:

- A fully constructed 3D maze environment
- First-person navigation using keyboard and mouse
- Real-time rendering of the environment
- Collision detection preventing invalid movement
- Goal detection mechanism for game completion
- Timer system to track performance

6.2 User Interaction

The player can:

- Move freely inside the maze
- Look around using mouse controls
- Navigate through paths while avoiding walls
- Reach the goal to complete the game

The controls are responsive and provide a smooth gameplay experience.

6.3 Visual Output

The environment includes:

- 3D walls and floor structures
- Lighting effects for realism
- Materials and textures for visual enhancement
- Features like minimap and flashlight

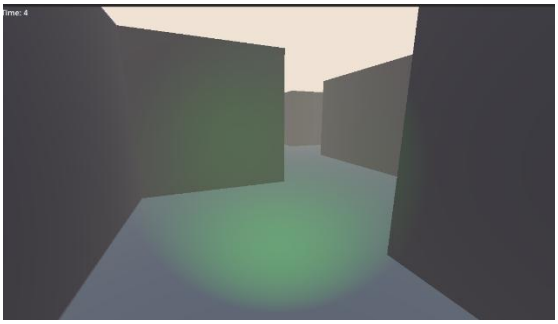

Fig (i): Landing UI

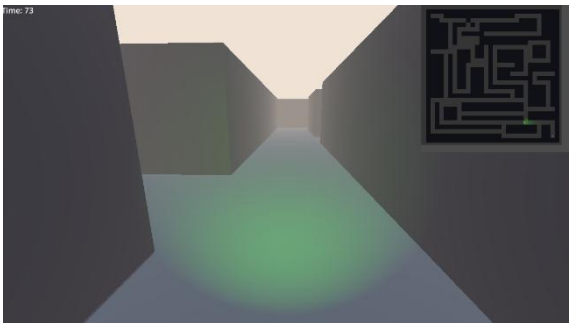

fig (ii): Minimap UI

fig (iii): Escaped Page

6.4 Performance Evaluation

- The game runs smoothly in real time
- No significant lag or frame drops observed
- Efficient handling of physics and rendering

6.5 Sample Output Description

When the game starts:

1. The player spawns inside the maze
2. Timer begins automatically
3. Player navigates using controls
4. Upon reaching the goal:
    - Timer stops
    - Completion message is displayed

6.6 Learning Outcome

The output demonstrates successful implementation of:

- 3D coordinate mapping
- Transformations and camera systems
- Physics and collision detection
- Real-time rendering pipeline

Discussion

The development of the First-Person Maze Game provided valuable practical insight into the application of core computer graphics concepts in a real-time interactive system. The project successfully integrates multiple components such as 3D modeling, transformation, rendering, physics, and user interaction into a cohesive application.

One of the key observations during the project was the importance of coordinate system understanding and transformation techniques. The mapping of a discrete 2D maze grid into a continuous 3D environment required careful handling of spatial relationships. This helped reinforce theoretical concepts such as coordinate transformations, object positioning, and scaling.

The implementation of the first-person camera system highlighted the role of viewing transformations in computer graphics. Separating the camera rotation into yaw and pitch components ensured realistic navigation and prevented issues such as unnatural rotations. This closely relates to real-world FPS camera systems used in modern games.

Another important aspect was collision detection and physics handling. By using the built-in physics engine, the player movement was restricted within the maze boundaries, ensuring logical interaction with the environment. This demonstrated how object-space calculations and physics simulations contribute to realism in graphics applications.

The project also emphasized the significance of the graphics rendering pipeline. Although much of the rendering process is handled internally by the engine, understanding how geometric data is transformed into visual output provided a deeper appreciation of rendering techniques such as lighting, shading, and material application.

From a software engineering perspective, the use of a modular design approach proved highly effective. Dividing the system into independent components (maze, player, UI, environment) allowed parallel development and simplified debugging. This approach also ensured scalability and ease of future enhancements.

However, certain limitations were observed. The maze structure is currently static, which reduces replayability. Additionally, the absence of advanced features such as AI-based enemies, procedural

generation, and dynamic lighting limits the complexity of the system. Performance optimization for larger environments was also not extensively explored.

Overall, the project demonstrates how theoretical concepts of computer graphics can be effectively translated into a functional and interactive application. It bridges the gap between academic learning and practical implementation, providing a strong foundation for further exploration in graphics and game development.

Conclusion

The First-Person Maze Game project successfully achieves its objective of demonstrating the practical application of fundamental computer graphics concepts in an interactive 3D environment. By utilizing the **Godot Engine**, the project integrates key elements such as 3D modeling, camera systems, transformation techniques, collision detection, and real-time rendering into a cohesive system.

The project not only reinforces theoretical knowledge but also enhances practical skills in game development, problem-solving, and system design. The implementation of a first-person navigation system and a structured maze environment highlights the effective use of coordinate mapping, physics handling, and rendering pipelines.

Additionally, the collaborative nature of the project contributed to the development of teamwork and project management skills. The modular architecture adopted during development ensured efficient integration and maintainability of the system.

Despite certain limitations, the project provides a solid foundation for further improvements and extensions. It serves as an important learning experience and demonstrates the potential of computer graphics in creating immersive and interactive applications.

In conclusion, the project successfully bridges the gap between theory and practice, offering both technical understanding and hands-on experience in the field of computer graphics and interactive system design.

Future Work

• Add multiple levels of difficulty.

• Implement dynamic lighting.

• Add enemies or obstacles.

• Improve maze complexity.

References

1. Godot Engine Documentation. (2024). 3D Graphics and Physics. [Online]. Available: https://docs.godotengine.org/

2. Hughes, J. F., et al. (2013). Computer Graphics: Principles and Practice. Addison-Wesley Professional.

3. Buck, J. (2015). The Buckblog: Maze Generation: Recursive Backtracking. [Online].

4. IOE Thapathali Campus. Computer Graphics & Visualization Course Manual.